CHAPTER 1

PUB-SUB

1 System Design: The Pub-Sub Abstraction 2
2 Introduction to Pub-Sub 2
3 Design of a Pub-Sub System 6

Learn the design of a pub-sub system, focusing on how to implement and optimize this communication pattern in distributed systems.

1 System Design: The Pub-Sub Abstraction

1.1 What is a pub-sub system?

Publish-subscribe messaging, often known as **pub-sub messaging**, is an asynchronous service-to-service communication method that's popular in serverless and microservices architectures. Messages can be sent asynchronously to different subsystems of a system using the pub-sub system. All the services subscribed to the pub-sub model receive the message that's pushed into the system.
For example, when a famous athlete posts on Instagram or shares a tweet, all of their followers are updated. Here , the athlete is the publisher, their post or tweet is the message, and all of their followers are subscribers.

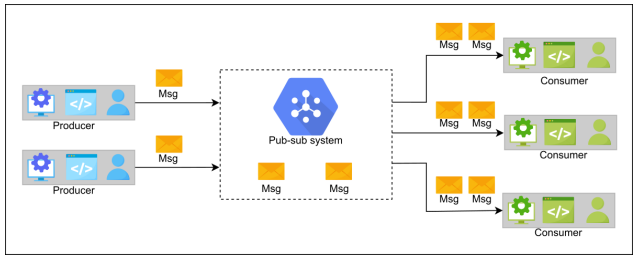


Figure 1.1: The pub-sub system

1.2 Motivation

The hardware infrastructure of distributed systems consists of millions of machines. Using a pub-sub system to communicate asynchronously increases scalability. Producers and consumers are disconnected and operate independently, thereby allowing us to scale and develop them separately. The decoupling between components, producers and consumers, allows greater scalability because adding or removing any component doesn't affect the other components.

1.3 How do we design a pub-sub system?

We have divided the pub-sub system design into the following lessons:

1. **Introduction**: In this lesson, we learn about the use cases of the pub-sub system, define its requirements, and design the API for it.
2. **Design**: In this lesson, we discuss two designs of the pub-sub system, one with messaging queues and the other with a broker.

2 Introduction to Pub-Sub

Pub-sub messaging offers asynchronous communication. Let's explore the use cases where it is beneficial to have a pub-sub system.

2.1 Use cases of pub-sub

A few use cases of pub-sub are listed below:

- **Improved performance**: The pub-sub system enables push-based distribution, alleviating the need for message recipients to check for new information and changes regularly. It encourages faster response times and lowers the delivery latency.

- **Handling ingestion**: The pub-sub helps in handling log ingestion(Log ingestion refers to importing logs into a database for immediate use, such as monitoring. Huge applications like Facebook produce massive amounts of data and logs. Real-time fetching and processing of logs allow us to run analysis and take quick actions in case of failures.). The user-interaction data can help us figure out useful analyses about the behavior of users. We can ingest a large amount of data to the pub-sub system, so much so that it can deliver the data to any analytical system to understand the behavior patterns of users. Moreover, we can also log the details of the event that's happening while completing a request from the user. Large services like Meta use a pub-sub system called Scribe to know exactly who needs what data, and remove or archive processed or unwanted data. Doing this is necessary to manage an enormous amount of data.
- **Real-time monitoring**: Raw or processed messages of an application or system can be provided to multiple applications to monitor a system in real time.
- **Replicating data**: The pub-sub system can be used to distribute changes. For example, in a leader-follower protocol, the leader sends the changes to its followers via a pub-sub system. It allows followers to update their data asynchronously. The distributed caches can also refresh themselves by receiving the modifications asynchronously. Along the same lines, applications like WhatsApp that allow multiple views of the same conversation—for example, on a mobile phone and a computer's browser—can elegantly work using a pub-sub, where multiple views can act either as a publisher or a subscriber.

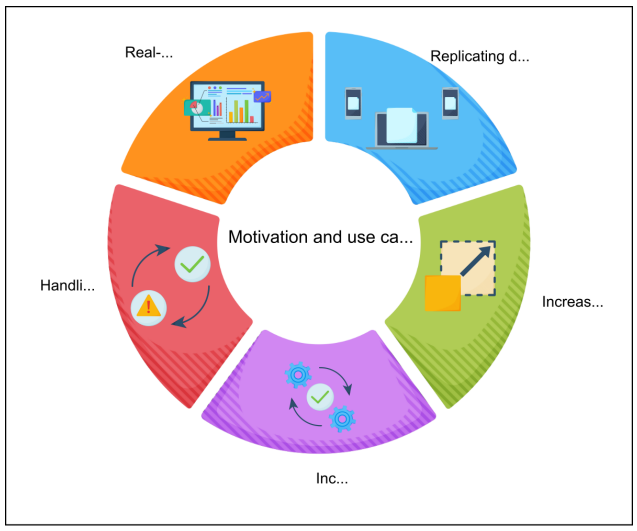


Figure 1.2: Motivation and use cases of the pub-sub system

Quiz: Question 1: What are the similarities and differences between a pub-sub system and queues? **Answer:** The pub-sub system and queues are similar because they deliver information that's produced by the producer to the consumer. The difference is that only one consumer consumes a message in the queue,

while there can be multiple consumers of the same message in a pub-sub system. **Question 2:** How are producers and consumers decoupled from one another in a pub-sub system? **Answer:** Producers don't know who'll end up reading their information. They just send it to the system, and it is read by the consumer. The system acts as a decoupling layer between the producers and consumers. Producers are not affected by slow consumers, the count of consumers, or the failure of consumers. We can scale them independently.

2.2 Requirements

We aim to design a pub-sub system that has the following requirements.

Functional requirements

Let's specify the functional requirements of a pub-sub system:

- **Create a topic**(A topic is a collection of related events or messages. The producers can create a topic and write the message into a topic. Consumers can subscribe to a topic and read from that topic.): The producer should be able to create a topic.
- **Write messages**: Producers should be able to write messages to the topic.
- **Subscription**: Consumers should be able to subscribe to the topic to receive messages.
- **Read messages**: The consumer should be able to read messages from the topic.
- **Specify retention time**: The consumers should be able to specify the retention time after which the message should be deleted from the system.
- **Delete messages**: A message should be deleted from the topic or system after a certain retention period as defined by the user of the system.

You're designing a real-time chat application that has channels for different discussions. Explain how you would use a pub-sub system for its messaging component.
Describe the mapping of the chatting application onto the following pub-sub components:

- Topics
- Subscribers
- How message delivery would work

Write your answer in the widget below.

Question:
How you would use a pub-sub system in a chat application?
Reference Answer:

1. Topics: Channels.
2. Subscribers: Users within the channel.
3. Message delivery: Publishers (users sending messages) publish to their topic. The pub-sub brokers broadcast messages to all topic subscribers.

Non-functional requirements

We consider the following non-functional requirements when designing a pub-sub system:

- **Scalable**: The system should scale with an increasing number of topics and increasing writing (by producers) and reading (by consumers) load.
- **Available**: The system should be highly available, so that producers can add their data and consumers can read data from it anytime.
- **Durability**: The system should be durable. Messages accepted from producers must not be lost and should be delivered to the intended subscribers.
- **Fault tolerance**: Our system should be able to operate in the event of failures.
- **Concurrent**: The system should handle concurrency issues where reading and writing are performed simultaneously.

2.3 API Design

We'll exclude some parameters from the functions below, such as the producer or consumer's identifier. Let 's assume that this information is available from the underlying connection context. The API design for this problem is as follows:

Create a topic

The API call to create a topic should look like this:

```
create(topic_ID, topic_name)
```

This function returns an acknowledgment if it successfully creates a topic, or an error if it fails to do so.

Parameter	Description
`topic_ID`	It uniquely identifies the topic.
`topic_name`	It contains the name of the topic.

Write a message

The API call to write into the pub-sub system should look like this:

```
write(topic_ID, message)
```

The API call will write a `message` into a topic with an ID of topic_ID. Each message can have a maximum size of 1 MB. This function will return an acknowledgment if it successfully places the data in the systems, or an appropriate error if it fails.

Parameter	Description
`message`	The message to be written in the system.

Read a message

The API call to read data from the system should look like this:

```
read(topic_ID)
```

The topic is found using topic_ID, and the call will return an object containing the message to the caller.

Parameter	Description
`topic_ID`	It is the ID of the topic against which the message will be read.

Subscribe to a topic

The API call to subscribe to a topic from the system should look like this:

```
subscribe(topic_ID)
```

The function adds the consumer as a subscriber to the topic that has the topic_ID.

Unsubscribe from a topic

The API call to unsubscribe from a topic from the system should look like this:

```
unsubscribe(topic_ID)
```

The function removes the consumer as a subscriber from the topic that has the topic_ID.

Parameter	Description
`topic_ID`	The ID of the topic to which the consumer will be subscribed.

Parameter	Description
`topic_ID`	The ID of the topic against which the consumers will be unsubscribed.

Delete a topic

The API call to delete a topic from the system should look like this:

```
delete_topic(topic_ID)
```

The function deletes the topic on the basis of the topic_ID.

Parameter	Description
`topic_ID`	The ID of the topic which is to be deleted.

2.4 Building blocks we will use

The design of pub-sub utilizes many building blocks that have been discussed in the initial chapters. We 'll consider the following lessons on building blocks.

```
create(topic_ID, topic_name)
```

- **Database**: We'll use databases to store information like subscription details.
- **Distributed messaging queue**: We'll use a messaging queue to store messages sent by the producer.
- **Key-value**: We'll use a key-value store to hold information about consumers.

In the next lesson, we'll focus on designing a pub-sub system.

3 Design of a Pub-Sub System

3.1 First design

In the previous lesson, we discussed that a producer writes into topics, and consumers subscribe to a topic to read messages from that topic. Since new messages are added at the end of the queue, we can use distributed messaging queues for topics.
The components we'll need have been listed below:

- **Topic queue**: Each topic will be a distributed messaging queue so we can store the messages sent to us from the producer. A producer will write their messages to that queue.
- **Database**: We'll use a relational database that will store the subscription details. For example, we need to store which consumer has subscribed to which topic so we can provide the consumers with their desired messages. We'll use a relational database since our consumer-related data is structured and we want to ensure our data integrity.
- **Message director**: This service will read the message from the topic queue, fetch the consumers from the database, and send the message to the consumer queue.
- **Consumer queue**: The message from the topic queue will be copied to the consumer's queue so the consumer can read the message. For each consumer, we'll define a separate distributed queue.

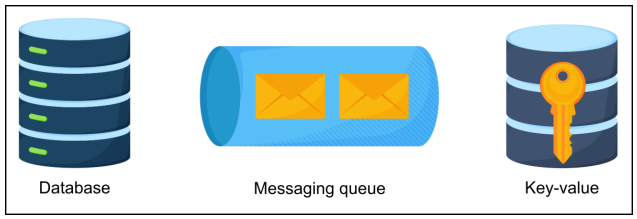


Figure 1.3: The building blocks we'll use

- **Subscriber**: When the consumer requests a subscription to a topic, this service will add an entry into the database.

The consumer will subscribe to a topic, and the system will add the subscriber's details to the database. The producer will write into the topics, and the message director will read the message from the queue, fetch the details to whom it should add the message, and send it to them. The consumers will consume the message from their queue.

> **Note**: We'll use fail-over services for the message director and subscriber to guard against failures.

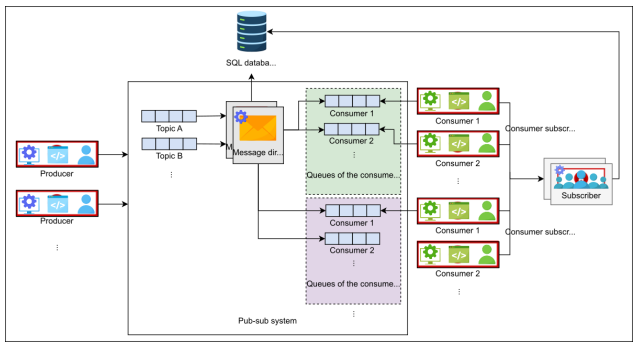


Figure 1.4: Using the distributed messaging queue

Using the distributed messaging queues makes our design simple. However, the huge number of queues needed is a significant concern. If we have millions of subscribers for thousands of topics, defining and maintaining millions of queues is expensive. Moreover , we'll copy the same message for a topic in all subscriber queues, which is unnecessary duplication and takes up space.

> **Quiz: Question 1:** Is there a way to avoid maintaining a separate queue for each reader? **Answer:** In messaging queues, the message disappears after the reader consumes it. So, what if we add a counter for each message? The counter value decrements as a subscriber consumes the message. It does not delete the message until the counter becomes zero. Now, we don't need to keep a separate queue for each reader. **Question 2:** What is the problem with the previous approach? **Answer:** The unread messages can become a bottleneck if we use the conventional queue API. For example, if 9 out of 10 readers have consumed the message present at the start of the queue, then that message won't be deleted until the tenth consumer has also consumed the message, and the first nine consumers won't be able to move forward. We'll need to change the storage interface so that consumers can independently consume data. Our system will need to keep sufficient metadata and track what information each consumer has consumed and delete a message when the information has been consumed by all consumers. It resembles with reference count mechanism in Linux's hard link of files.

3.2 Second design

Let's consider another approach to designing a pub-sub system.

High-level design

At a high level, the pub-sub system will have the following components:

- **Broker**: This server will handle the messages. It will store the messages sent from the producer and allow the consumers to read them.
- **Cluster manager**: We'll have numerous broker servers to cater to our scalability needs. We need a cluster manager to supervise the broker's health. It will notify us if a broker fails.
- **Storage**: We'll use a relational database to store consumer details, such as subscription information and retention period.
- **Consumer manager**: This is responsible for managing the consumers. For example, it will verify if the consumer is authorized to read a message from a certain topic or not.

Besides these components, we also have the following design considerations:

- **Acknowledgment**: An acknowledgment is used to notify the producer that the received message has been stored successfully. The system will wait for an acknowledgment from the consumer if it has successfully consumed the message.
- **Retention time**: The consumers can specify the retention period time of their messages. The default will be seven days, but it is configurable. Some applications like banking applications require the data to be stored for a few weeks as a business requirement, while an analytical application might not need the data after consumption.

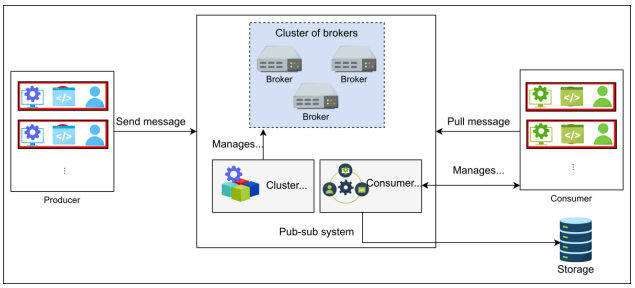


Figure 1.5: High-level design of a pub-sub system

Let's understand the role of each component in detail.

3.3 Broker

The broker server is the core component of our pub-sub system. It will handle write and read requests. A broker will have multiple topics, where each topic can have multiple **partitions** associated with it. We use partitions to store messages in the local storage for persistence. Consequently, this improves availability. Partitions contain messages encapsulated in **segments**.
Segments help identify the start and end of a message using an offset address. Using segments, consumers consume the message of their choice from a partition by reading from a specific offset address. The illustration below depicts the concept that has been described above.

As we know, a **topic** is a persistent sequence of messages stored in the local storage of the broker. Once the data has been added to the topic, it cannot be modified. Reading and writing a message from or to a topic is an I/O task for computers, and scaling such tasks is challenging. This is the reason we split the topics into multiple partitions. The data belonging to a single topic can be present in numerous partitions. For example, let's assume we have Topic A and we allocate three partitions for it. The producers will send their messages to the relevant topic. The messages received will be sent to various partitions on the basis of the round-robin algorithm(Round-robin is a simple and starvation-free CPU scheduling algorithm. In

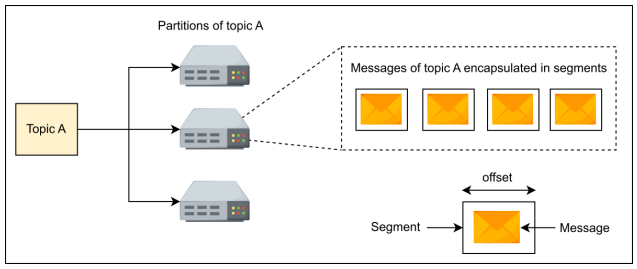
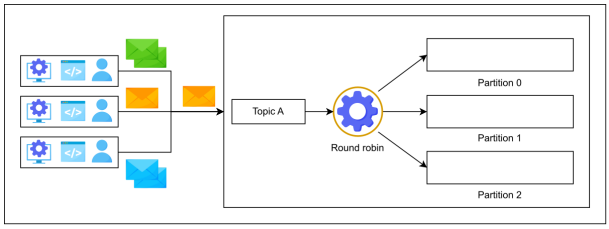
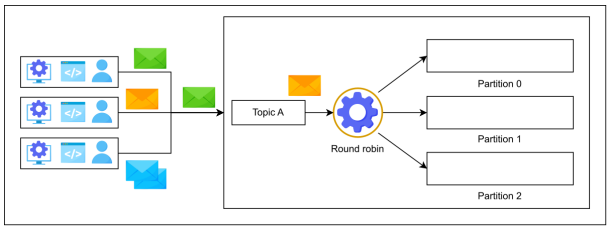


Figure 1.6: A depiction of how messages are stored within segments inside a partition

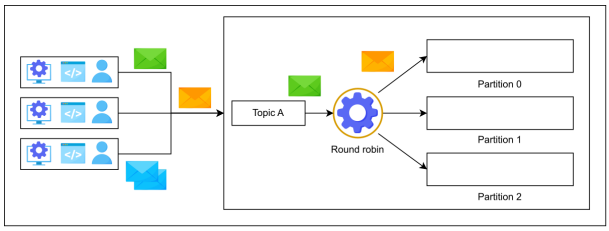
this algorithm, each process or job is given a fixed amount of time slice cyclically. Consider an example in which we have the time slice of 200 ms and a job whose execution time is 1000 ms. The operating system (OS) assigns the CPU to the task for 200 ms in the round-robin algorithm and then puts the job at the end of the scheduling queue. The other jobs in the queue are given equal time slices (200 ms each). This approach continues until all the jobs finish their execution.). We'll use a variation of round-robin: weighted round-robin(Weighted round robin is a generalization of round-robin scheduling. It serves a set of queues or tasks. Whereas round-robin cycles over the queues or tasks and gives one service opportunity per cycle, a weighted round-robin offers to each a fixed number of opportunities as specified by the configured weight, which serves to influence the portion of capacity received by each queue or task. In computer networks, a service opportunity is the emission of one packet if the selected queue is non-empty. Source: Wikipedia). The following slides show how the messages are stored in various partitions belonging to a single topic.



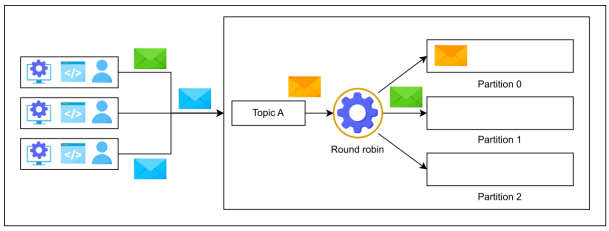
(a) The messages are added to the relative partitions.



(b) The round-robin algorithm decides the partition where we need to store the message.



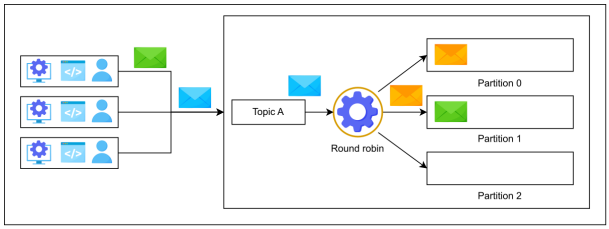
(a) The round-robin algorithm sends the message to Partition 0 and decides the partition for the incoming message.



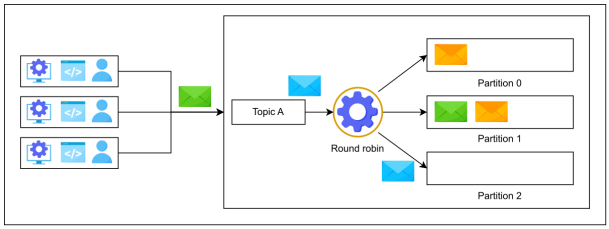
(b) The round-robin algorithm sends the message to Partition 1 and decides the partition for the incoming message.

Quiz: Question 1: Strict ordering ensures that the messages are stored in the order in which they are produced. How can we ensure strict ordering for our messages? **Answer:** We'll assign each partition a unique ID, 'partition_ID'. The user can provide the 'partition_ID' while writing into the system. In this way,

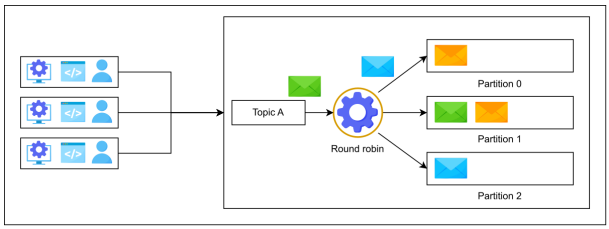
the messages will be sent to the specified partition, and the ordering will be strict. Our API call to write into the pub-sub system looks like this: "'txt write(topic_ID, partition_ID, message) "' If the user does not provide the 'partition_ID', we'll use the weighted round-robin algorithm to



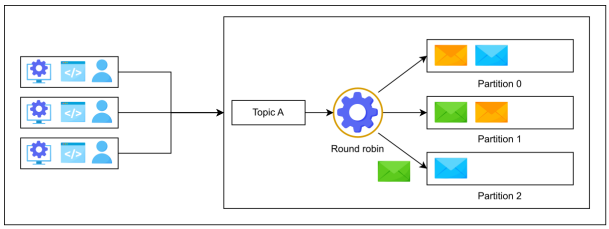
(a) The round-robin algorithm sends the message to Partition 1 and decides the partition for the incoming message.



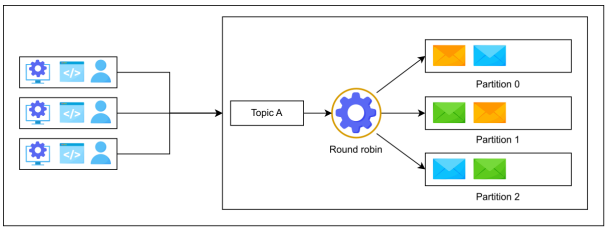
(b) The round-robin algorithm sends the message to Partition 2 and decides the partition for the incoming message.



(a) The round-robin algorithm sends the message to Partition 0 and decides the partition for the incoming message.



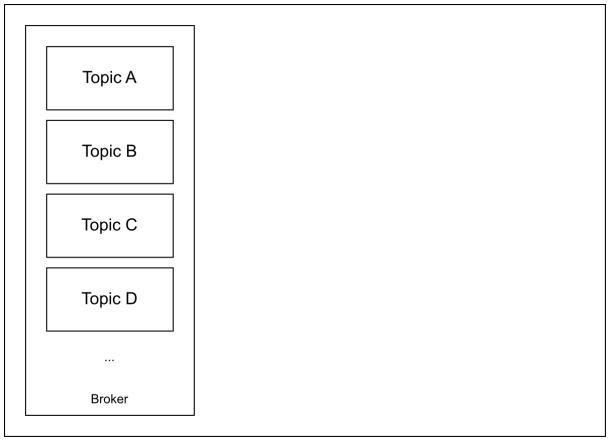
(b) The round-robin algorithm sends the message to Partition 2 and decides the partition for the incoming message.



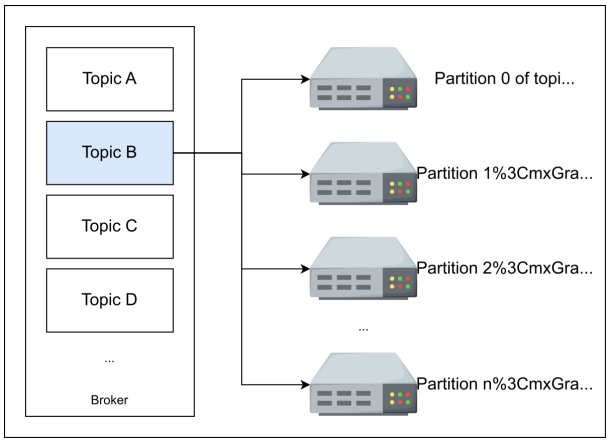
(a) The messages are added to the relative partitions.

decide which message has to be sent to which partition. It might seem strange to give the ability to choose a partition to the client of pub-sub. However, such a facility can be the basis from where clients can get data for some specific time period—for example, getting data from yesterday. For simplicity, we won't include time-based reading in our design. **Question 2:** What problems can arise if all partitions are on the same broker? **Answer:** If the broker fails or dies, all the messages in the partitions will be lost. To avoid this, we need to make sure that the partitions are spread on different brokers. **Question 3:** Why can't we use blob stores like S3 to keep messages, instead of the broker's local storage? **Answer:** Blob stores like S3 are not optimized for writing and reading short-sized data. If our data is geo-replicated, the above problem is exacerbated. Therefore, we used the server's local persistent store with append-based writing. Traditional hard disks are specially tuned to provide good write performance with writing to contiguous tracks or sectors. Reading throughput and latency is also good for contiguous regions of the disk because it allows extensive data caching. **Question 4:** If we use a round-robin algorithm to send messages to a partition, how does the system know where to look when it is time to read? **Answer:** Our system will need to keep appropriate metadata persistently. This metadata will keep mappings between the logical index of segment or messages to the server identity or partition identifier. We'll discuss consumer manager later in the lesson, which will keep the required information.

We'll allocate the partitions to various brokers in the system. This just means that different partitions of the same topic will be in different brokers. We 'll follow strict ordering in partitions by adding newer content at the end of existing messages.
Consider the slides below. We have various brokers in our system. Each broker has different topics. The topic is divided into multiple partitions.



(a) A broker contains multiple topics.



(b) Physical view: A topic is split into numerous partitions that are stored on other brokers.

We discussed that a message will be stored in a segment. We'll identify each segment using an offset. Since these are immutable records, the readers are independent and they can read messages anywhere from this file using the necessary API functions. The following slides show the segment level detail.

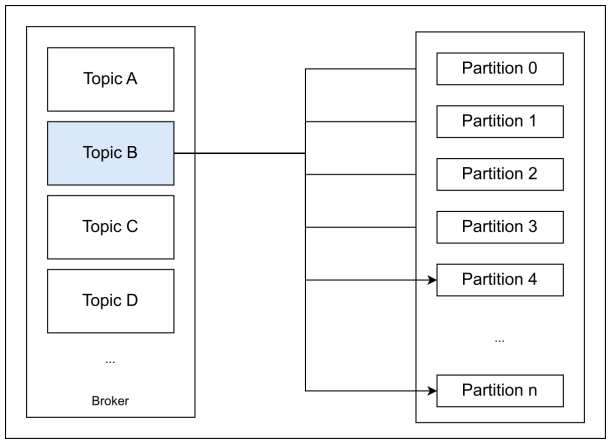
The broker solved the problems that our first design had. We avoided a large number of queues by partitioning the topic. We introduced parallelism using partitions that avoided bottlenecks while consuming the message.
How would you design a pub-sub system to ensure consumers receive messages in the correct order, even when subscribing to multiple topics or partitions, assuming messages have different time stamps?

Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

How do we ensure message order in a pub-sub?

Reference Answer:



(a) Logical view: We can represent the partitions in another way. Here, Topic B is split into multiple partitions.

(a) A new entry will be added at the end of the file.

(b) The consumers can read anywhere from the file. Producers add to the end of file.

To ensure consumers receive messages in the correct order, even with multiple topics, partitions, and varying timestamps:

1. Partitioning strategy: Assign each topic to its own partition, ensuring ordered message delivery within each partition.
2. Timestamp-based ordering: Include timestamps in each message and have consumers process messages based on these timestamps to maintain global order across partitions or topics.
3. Handling multiple topics: Maintain separate consumer queues for each topic, or merge queues while respecting timestamps to preserve order across topics.
4. Cross-partition synchronization: Use a coordinator system to track and synchronize message order across multiple partitions based on timestamps.

This approach ensures correct message ordering in a system with multiple topics and partitions.

3.4 Cluster manager

We'll have multiple brokers in our cluster. The cluster manager will perform the following tasks:

- **Broker and topics registry**: This stores the list of topics for each broker.
- **Manage replication**: The cluster manager manages replication by using the leader-follower approach. One of the brokers is the leader. If it fails, the manager decides who the next leader is. In case the follower fails, it adds a new broker and makes sure to turn it into an updated follower. It updates the metadata accordingly. We'll keep three replicas of each partition on different brokers.
- **Authorization**: The cluster manager handles authorization for broker and topic access, as well as controlling message replication across clusters.

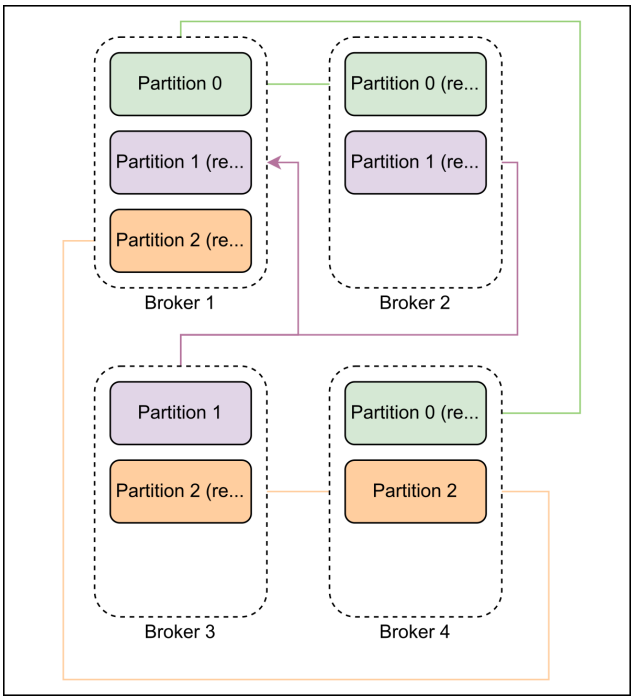


Figure 1.15: Replication at the partitioning level

3.5 Consumer manager

The consumer manager will manage the consumers. It has the following responsibilities:

- **Verify the consumer**: The manager will fetch the data from the database and verify if the consumer is allowed to read a certain message. For example, if the consumer has subscribed to Topic A (but not to Topic B), then it should not be allowed to read from Topic B. The consumer manager verifies the consumer's request.
- **Retention time management**: The manager will also verify if the consumer is allowed to read the specific message or not. If, according to its retention time, the message should be inaccessible to the consumer, then it will not allow the consumer to read the message.
- **Message receiving options management**: There are two methods for consumers to get data. The first is that our system pushes the data to its consumers. This method may result in overloading the consumers with continuous messages. Another approach is for consumers to request the system to read data from a specific topic. The drawback is that a few consumers might want to know about a message as soon as it is published, but we do not support this function. Therefore, we'll support both techniques. Each consumer will inform the broker that it wants the data to be pushed automatically or it needs the data to read itself. We can avoid overloading the consumer and also provide liberty to the consumer. We'll save this information in the relational database along with other consumer details.
- **Allow multiple reads**: The consumer manager stores the offset information of each consumer. We'll use a key-value to store offset information against each consumer. It allows fast fetching and increases the availability of the consumers. If Consumer 1 has read from offset 0 and has sent the acknowledgment, we'll store it. So, when the consumer wants to read again, we can provide the next offset to the reader for reading the message.

3.6 Finalized design

The finalized design of our pub-sub system is shown below.

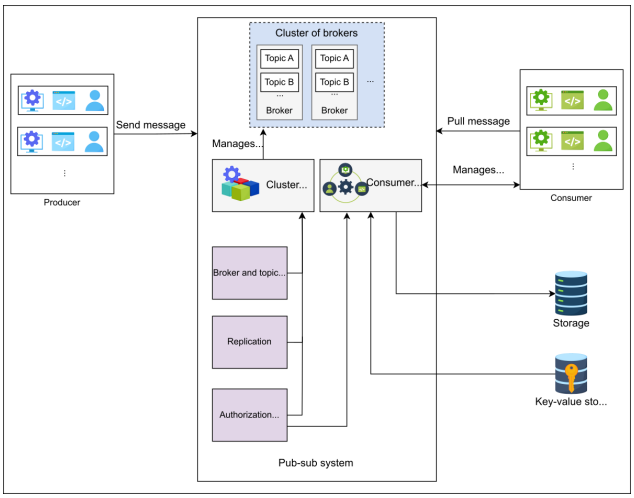


Figure 1.16: High-level design of a pub-sub system

3.7 Conclusion

We saw two designs of pub-sub, using queues and using our custom storage optimized for writing and reading small-sized data.
There are numerous use cases of a pub-sub. Due to decoupling between producers and consumers, the system can scale dynamically, and the failures are well-contained. Additionally , due to proper accounting of data consumption, the pub-sub is a system of choice for a large-scale system that produces enormous data. We can determine precisely which data is needed and not needed.